TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

Minor Project Proposal

ON

GEO-LOCATING USERS IN MOUNTAINS BY DETECTING SKYLINES IN CAPTURED PHOTOS

Submitted By:

Bibha Ray	(THA080BCT009)
Prasiddha Raj Poudel	(THA080BCT027)
Reema Kasula	(THA080BCT032)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

June 2026

ABSTRACT

Navigation in Nepal's Khumbu region is often difficult when global positioning system signals fail in deep mountain valleys due to topographic shielding. This project proposes a visual positioning system that uses the mountain skyline to determine a user's location. The system will extract the silhouette from user's photograph and match them against a database of synthetic horizons. These reference views are generated using digital elevation models of the Himalayan landscape. A two-stage matching engine combining sliding-window Euclidean distance and Dynamic Time Warping (DTW) can resolve the coordinates even when the camera heading is unknown. This framework aims to provide a navigation alternative for travelers in remote environments.

***Keywords*—**Visual Geolocation, Skyline Localization, Digital Elevation Models, Himalayan Topography, Sliding Window Search, Dynamic Time Warping, Khumbu Region.

TABLE OF CONTENTS

1 LIST OF FIGURES

2 INTRODUCTION

2.1 Background

Geographic positioning in the Himalayas is often unreliable due to GNSS signal blockage in deep valleys. However, the mountain skyline provides a stable geometric reference for navigation [1]. A camera-based system can determine location by matching photo skylines against a database of horizons generated from Digital Elevation Models (DEMs) [1,2].

2.2 Motivation

In high-altitude regions like Khumbu, navigation failure can be life-threatening. Since portable devices are common, using their cameras for offline positioning provides a robust backup to satellite systems. This technology is essential for trekkers and search-and-rescue teams operating in areas with poor signal reception or GNSS denial.

2.3 Problem Statement

The objective is to estimate a camera's location and orientation using only the visible mountain skyline. The system must address several real-world challenges:

1. **Extraction:** Haze, clouds, and low contrast frequently obscure the skyline boundary in real-world photographs [3].
2. **Search Scale:** Matching a partial skyline against a massive 360-degree database is computationally expensive without prior orientation data [1].
3. **Data Scarcity:** Annotated mountain photographs are scarce for the Himalayas, requiring the use of synthetic terrain views for training extraction models [4].

2.4 Objectives

The core objectives are:

- To generate a 360-degree horizon database from high-resolution elevation models.
- To implement a matching pipeline using Euclidean distance and Dynamic Time Warping (DTW) to resolve geographic coordinates from photos.

2.5 Scope and Limitations

The study focuses on a 36.4 km $\times$ 31.2 km area in the Khumbu region of Nepal.

- **Northwest (NW):** 86.600°E, 28.050°N
- **Northeast (NE):** 86.970°E, 28.050°N

- **Southwest (SW):** 86.600°E, 27.770°N
- **Southeast (SE):** 86.970°E, 27.770°N

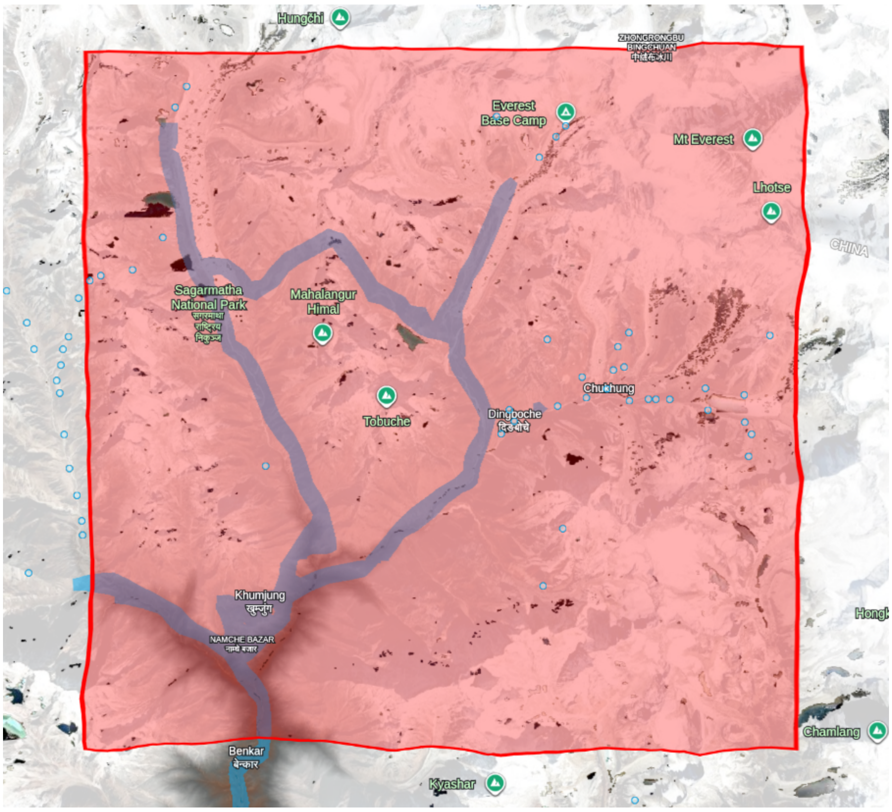

Figure 2-1 Region of Interest (ROI)

The system is designed for offline use on standard portable hardware. Limitations include a focus on primary skylines and failures in cases where weather or obstacles completely block the view.

3 LITERATURE REVIEW

3.1 Evolution of Terrain Representation

Terrain representation has evolved significantly over the last three decades. Early systems relied on manual surveying and simple topographic maps. These methods lacked the detail needed for automated navigation.

In 2000, the Shuttle Radar Topography Mission (SRTM) provided the first near-global elevation dataset [5]. Shortly after, the ASTER Global Digital Elevation Model (GDEM) was released in 2009 [1]. These legacy datasets provided crucial elevation grids. However, they exhibited significant errors in high-altitude areas [5]. Steep cliffs and deep valleys caused radar distortions [1, 5]. These distortions led to inaccurate peak heights [1].

To address these errors, researchers evaluated newer satellite datasets. In 2020, the European Space Agency released the Copernicus GLO-30 model. Recent evaluations confirm its superior vertical accuracy [5]. It out-performs older models in steep mountain terrain [5]. At 30-meter resolution, this model captures the unique shape of Himalayan peaks. The resulting database is small enough to run on portable devices. This makes it an ideal foundation for modern terrain-based positioning systems [1, 2].

3.2 Developments in Skyline Segmentation

Extracting a clear skyline from a photograph is a well-studied problem. Early approaches in the 2000s used classic edge-detection filters. These filters detected sudden changes in image intensity.

However, traditional edge-detection failed in real-world scenarios. Haze, clouds, and shadows often obscured the skyline. This led to broken or incorrect boundaries [1].

To solve this issue, researchers transitioned to machine learning. In 2015, convolutional neural networks revolutionized image segmentation. These deep models could identify the sky even in poor lighting conditions.

However, early neural networks were too heavy for mobile phones. In response, researchers developed compact architectures. MobileUNET became a popular choice for mobile devices. It simplifies calculations to save battery power while maintaining high pixel accuracy.

Other modern models adapt object detection frameworks. For example, YUNet adapts the YOLOv11 framework for rapid sky detection [3]. It achieves high segmentation scores under difficult weather conditions [3].

A persistent challenge was the lack of labeled training images. In 2017, Brejcha and Cadik introduced the GeoPose3K dataset [4]. This library contains over 3,000 precisely labeled mountain landscapes [4]. In 2022, Tomesek et al. explored cross-modal rendering to generate more training data [6]. These datasets allow modern networks to learn real-world features successfully.

3.3 Progress in Silhouette Matching and Heading Resolution

Once the skyline is extracted, it must be matched against the terrain database. This step calculates the camera's location and direction [1].

Historically, matching relied on simple pixel-by-pixel comparisons. This method was slow and easily confused by different camera lenses. It also required prior knowledge of the camera's direction.

To resolve this, researchers implemented a rotational sliding-window search [1]. The system compares the query photo against a 360-degree virtual horizon at multiple positions [1].

To make this process run in real time, researchers developed two-stage search systems. In 2013, Rakthanmanon et al. demonstrated fast subsequence matching under Dynamic Time Warping (DTW) [7]. DTW is a robust shape-alignment technique. It adjusts for perspective distortions caused by different camera lenses [7].

Running DTW across a large database remains computationally expensive [7]. To solve this bottleneck, Mueen et al. introduced a fast search algorithm in 2022 [8]. This algorithm performs rapid pruning by computing simple distance profiles [8]. It quickly filters out bad candidates. This leaves only a few profiles for DTW alignment. This combination enables fast, real-time location matching on mobile hardware.

3.4 Justification for GNSS-Based Validation

Developing an alternative positioning system requires a reliable testing standard. Researchers traditionally use high-accuracy GPS data to validate their systems.

Early positioning studies relied on small, custom photo sets. These sets lacked diverse lighting conditions and precise coordinates.

This issue was resolved with the release of the Google Street View Trekker dataset [9]. In 2014, researchers captured over 45,000 high-resolution panoramas along the Everest trail [9]. The Trekker hardware uses a multi-lens camera array. It also integrates precise satellite receivers and motion sensors. This hardware ensures highly accurate location tags for every image [9].

This georeferenced dataset provides an ideal reference map for our project. It contains thousands of real-world mountain views with verified coordinates. We use this dataset to measure our system's accuracy in the actual Khumbu environment.

4 METHODOLOGY

4.1 Proposed Approach and Tools

GNSS signals often fail in deep Himalayan valleys. Visual odometry drifts over long distances. Skyline matching uses stable mountain peaks as fixed landmarks. It works offline without cell service. This method is more reliable than radio-based navigation.

The system uses the Copernicus 30-meter Digital Elevation Model (DEM). Python is the main language for development. We use the FAISS library for fast vector search. The final deployment target is standard Android portable hardware.

4.2 System Design and Mathematical Foundation

The system consists of three main parts. First, we build a reference database from the DEM. Second, a neural network finds the skyline in photos. Third, we match the photo skyline to the database.

We represent the horizon as a 1D vector of elevation angles.

$$V = \{h_1, h_2, \ldots, h_{1440}\} \tag{4-1}$$

Each value represents a 0.25° azimuth step. We assume the earth is a sphere for local ray-casting. Atmospheric refraction is modeled as a constant factor.

4.3 Algorithms and Data Processing

We generate synthetic horizons via ray-casting. Real-world testing uses Google Street View photospheres. These images provide ground-truth GPS data. We resize all input photos to 512×512 pixels.

We use a two-phase search. Phase 1 uses FFT-based cross-correlation for speed. It finds the top 10 candidate locations. Phase 2 uses Dynamic Time Warping (DTW) for precision.

- **Search Complexity:** $O(m \log m)$ for Phase 1.
- **Optimization:** Sakoe-Chiba bands keep DTW linear.
- **Robustness:** Canny edge detection acts as a backup for the extraction model.

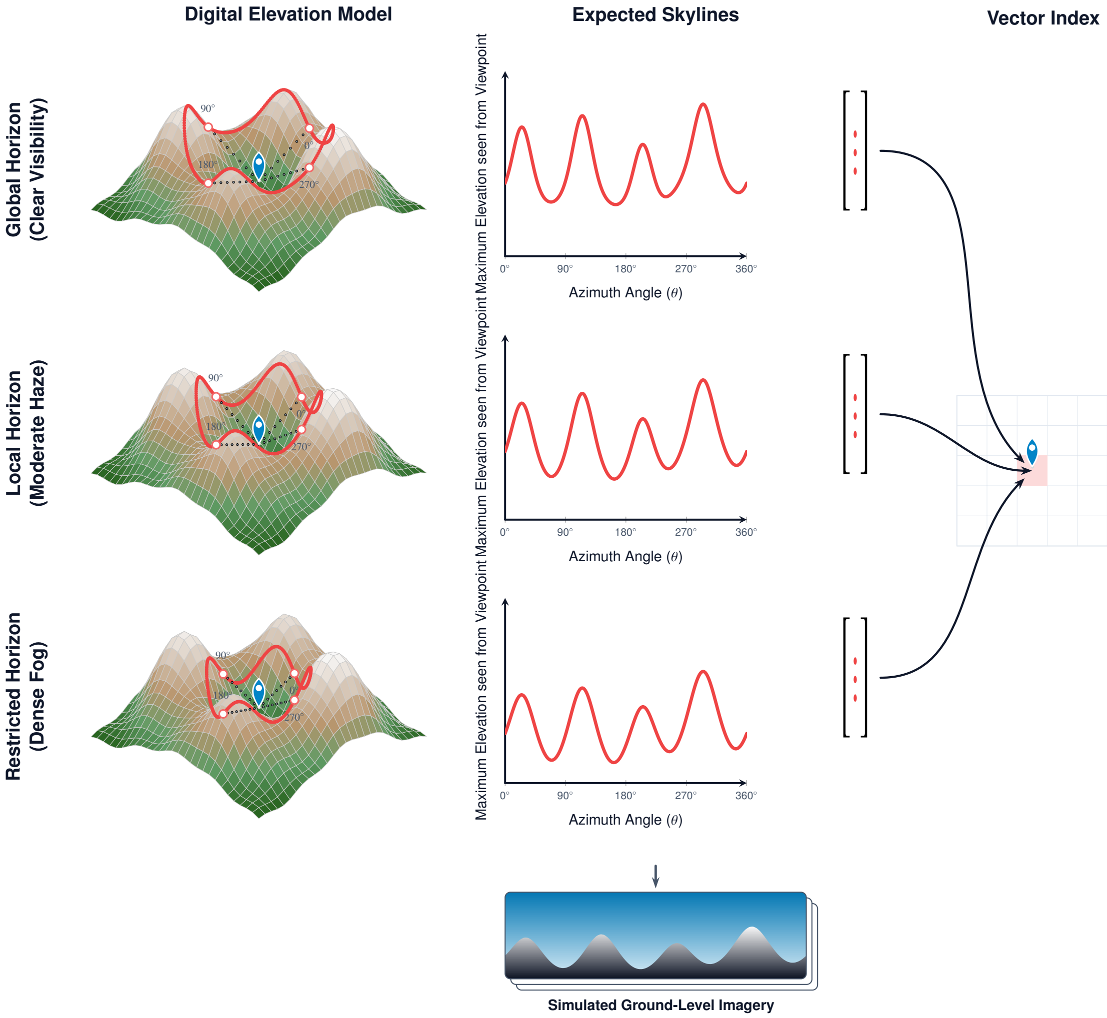

Figure 4-1 Workflow for database generation and skyline matching.

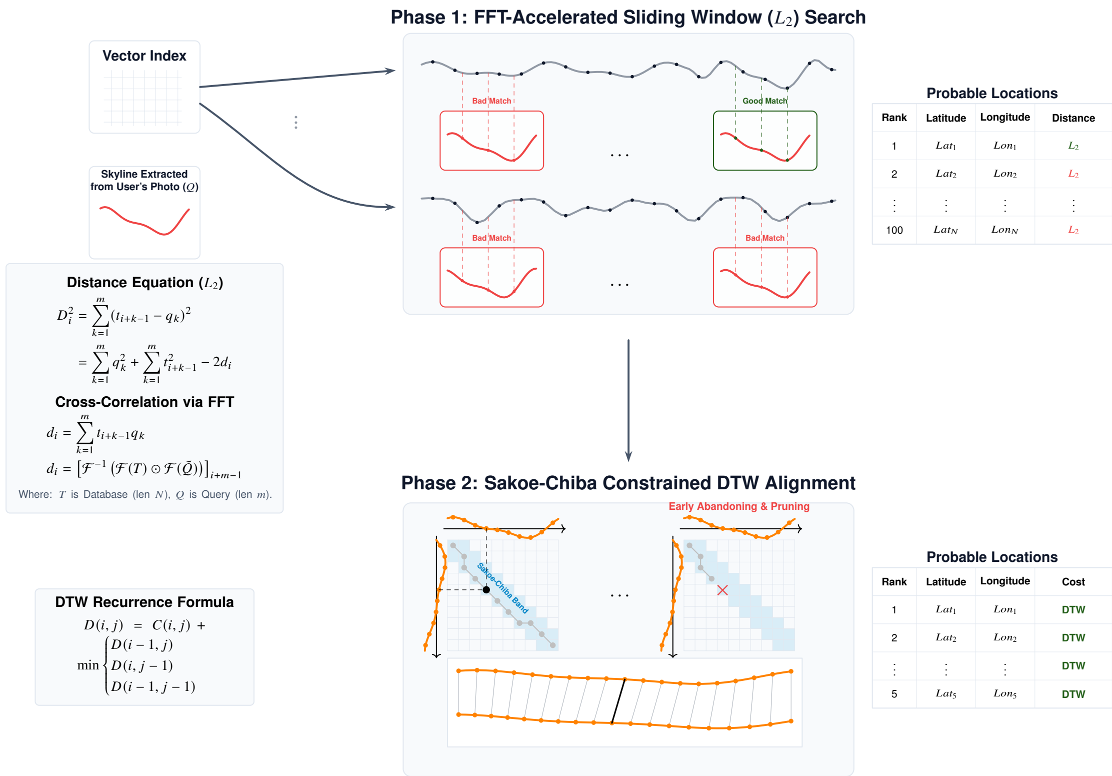

Figure 4-2 Detailed architecture of the matching engine.

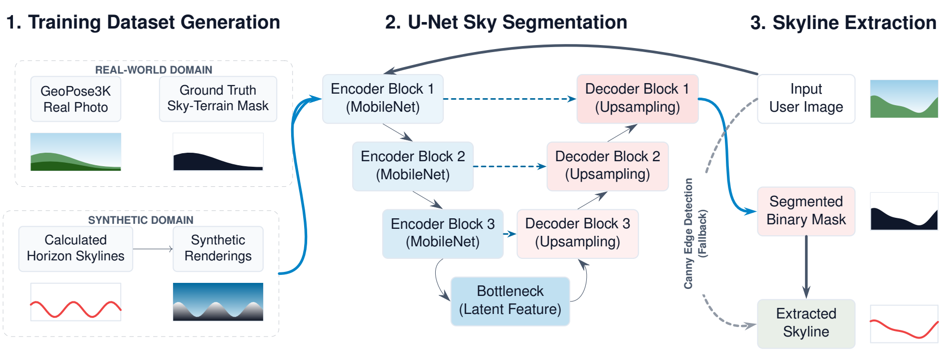

Figure 4-3 Pipeline for skyline extraction from photographs.

5 EXPECTED RESULTS

The project aims to deliver a functional prototype for offline geolocation. The system will extract mountain skylines from user photos and match them against a pre-computed database. Users can view the predicted location on a map alongside the top five matching candidates. Visual overlays will show the extracted skyline and corresponding database horizon to help verify results. Final testing uses Google Street View data to measure the system's effectiveness across the Khumbu trail network.

6 FEASIBILITY ANALYSIS

6.1 Overview

The feasibility analysis focuses on data acquisition, computational viability, and empirical validation. The system architecture generates a reference database of synthetic skyline profiles from a digital elevation model. These profiles are matched against query images using a two-stage search pipeline. Validation is performed using a systematic train/test framework on geotagged perspective imagery.

6.2 Elevation Data and Database Generation

6.2.1 Choice of DEM

Digital Elevation Models (DEMs) model the terrain surface as a grid of elevation values. This grid is used to reconstruct the 3D mountain horizons required for matching.

We selected the Copernicus GLO-30 DEM, which offers global coverage at a 30 m spatial resolution. In rugged alpine regions, GLO-30 has a measured root-mean-square vertical error of approximately 2.34 m [5]. Comparative evaluations in high-relief mountain ranges show that GLO-30 provides a more consistent and reliable terrain surface than older alternatives such as AW3D30 [10]. Additionally, GLO-30 has been successfully used in Himalayan glacier studies to track surface elevation changes over time [11], confirming its utility in high-altitude environments.

Other standard elevation models were considered but presented limitations for this application:

- **SRTM and NASADEM:** Contain data gaps in steep, rugged areas due to radar shadow and layover effects.
- **FABDEM:** Designed to filter out forest canopy and buildings. Because the high-altitude terrain of the Khumbu region contains negligible vegetation and structural cover, FABDEM offers no practical advantage over GLO-30 [12].

6.2.2 Data Volume and Computational Requirements

The study area encompasses the primary trekking corridors in the Khumbu region. To prevent horizon clipping during ray casting, we include a 50 km buffer zone around the target area. This defines a total bounding box of approximately 140 km $\times$ 140 km. At a 30 m grid spacing, this region contains approximately 22 million elevation cells, requiring between 50 MB and 90 MB of storage. This represents a compact dataset that can be easily downloaded and stored.

The reference database is constructed by casting radial search vectors outward from a spatial grid of 6,400 viewpoints (spaced 500 m apart in an 80×80 grid). To ensure sufficient directional resolution, the horizon at each viewpoint is sampled at $0.25°$ intervals, yielding 1,440 rays per profile:

$$360°/0.25° = 1440 \text{ rays.}$$

At a terrain resolution of 30 m, casting rays up to a maximum distance of 50 km requires approximately 1,667 elevation checks per ray. Consequently, generating a single profile involves:

$$1440 \text{ rays} \times 1667 \text{ steps} \approx 2.4 \times 10^6 \text{ elevation checks.}$$

For the entire viewpoint grid, the generation phase requires:

$$6400 \times 2.4 \times 10^6 \approx 15.4 \times 10^9 \text{ elevation checks.}$$

Although this is a large number of checks, standard array-based processing allows a modern computer to complete the entire database generation in a few minutes [2].

6.2.3 Database Storage

Each synthesized skyline profile is stored as a vector of elevation angles. At $0.25°$ resolution, a profile consists of 1,440 single-precision floating-point values (4 bytes each):

$$1440 \times 4 \text{ bytes} = 5760 \text{ bytes} \approx 5.6\,\text{KB per profile.}$$

To handle changing weather conditions, we store three versions of each profile (representing clear, hazy, and foggy conditions):

$$5.6\,\text{KB} \times 3 \approx 16.9\,\text{KB per viewpoint.}$$

The complete database for all 6,400 viewpoints requires:

$$6400 \times 16.9\,\text{KB} \approx 105\,\text{MB}.$$

This dataset fits easily within standard system memory. Consequently, a simple, in-memory array search is sufficient for retrieval, eliminating the need for complex external database software.

6.3 Two-Stage Matching Pipeline

The matching pipeline processes query images in two stages: a fast global search followed by a fine-alignment verification.

6.3.1 Step 1: Fast Search with MASS and FFT

The query image provides a partial skyline of length m (for example, $m = 240$ data points, representing a 60° field of view at 0.25° resolution). Since the camera's orientation is unknown, the query must be evaluated against all possible angular rotations of the 6,400 reference database profiles of length $N = 1440$.

A brute-force sliding-window comparison across all rotations requires $O(N \times m)$ operations. To accelerate this search, we use the Mueen's Algorithm for Similarity Search (MASS), which utilizes the Fast Fourier Transform (FFT) to compute the distance profile in $O(N \log N)$ time [8]. This approach builds upon established methods for rapid pattern matching in sequential data [13, 14].

For a single reference profile, the computational operations required by each method are:

- **Brute-Force Search:**

$$N \times m = 1440 \times 240 = 345{,}600 \text{ operations.}$$

- **MASS (FFT-based):**

$$N \log_2 N \approx 1440 \times 10.49 \approx 15{,}106 \text{ operations.}$$

Evaluating all 6,400 reference profiles requires approximately 2.2 billion operations using a brute-force approach, compared to only 97 million operations using MASS. This represents an approximate 23-fold reduction in computational complexity, enabling the first-stage search to complete in under one second on standard hardware.

6.3.2 Step 2: Fine Check with Dynamic Time Warping

The fast-search stage identifies the top 50 to 100 candidate viewpoints. However, camera focal lengths and lens distortions can stretch or compress the captured skyline. To handle these geometric variations, we perform a second-stage verification using Dynamic Time Warping (DTW) [15]. This two-step search architecture is consistent with established terrain-based camera geolocation pipelines [1].

Standard DTW has a quadratic complexity of $O(m^2)$. To keep the search fast, we apply two constraints:

1. **Sakoe-Chiba Band:** The alignment path is restricted to a narrow diagonal constraint window of width w. Setting w to 10% of the query length ($w = 24$) reduces the matching complexity per candidate to $O(m \times w)$.
2. **Early Abandoning and Pruning (EAP):** During evaluation, paths that exceed the current minimum distance threshold are terminated early [7].

Using these optimizations, evaluating a single candidate requires:

$$m \times w = 240 \times 24 = 5760 \text{ operations.}$$

Evaluating 100 candidates requires 576,000 operations. This computational load is minimal and runs efficiently on modern processors.

6.4 Horizon Extraction and Validation

6.4.1 Sky Segmentation Model

To extract a clean skyline query from a photograph, the system must segment the image into "sky" and "non-sky" regions. We propose a U-Net style architecture with a MobileNet-V3 backbone, which is designed for fast execution on portable hardware [16]. A standard Canny edge-detection and horizon-extraction algorithm [17] is integrated as a fallback mechanism to ensure reliability if the deep-learning model fails or exceeds latency limits. Other modern horizon-detection models also offer useful benchmarks for comparison [3].

We will pre-train the model on the GeoPose3K dataset, which contains over 3,000 mountain images with known camera poses [4]. Because GeoPose3K consists of images from the European Alps, we will fine-tune the model on localized Khumbu terrain images to adapt to the region's distinct atmospheric and geological conditions [6].

6.4.2 Ground-Truth Verification

To evaluate the system under real-world conditions, we use geotagged panoramic imagery from the 2014 Google Street View campaign along the trekking route from Lukla to Everest Base Camp [9]. This collection contains over 45,000 panoramas with verified geographic coordinates and orientation metadata.

To simulate standard perspective photography, we extract multiple perspective crops from each panorama (for example, generating 12 crops at 30° intervals, each representing a 60° horizontal field of view). The validation framework incorporates the following design choices:

- **Data Leakage Prevention:** The dataset is partitioned into training and testing sets at the panorama level rather than the crop level. This ensures that overlapping perspective views from the same physical coordinate do not appear in both subsets.
- **Geometric Distortion Correction:** Perspective crops are re-projected from the equirectangular panorama format into planar perspective views to remove geometric bending.
- **Meteorological Evaluation:** Test images are categorized by atmospheric visibility (clear, hazy, foggy) to evaluate the system's corresponding multi-weather database profiles.

6.5 Implementation and Resource Allocation

6.5.1 System Specifications

Because the reference database has a compact memory footprint of approximately 105 MB, we bypass complex vector indexing libraries (such as FAISS). Instead, we use a direct in-memory array search, which simplifies deployment and avoids external software dependencies.

Given the 500 m grid spacing of the reference viewpoints, geolocation accuracy is evaluated based on correct grid cell assignment. Directional accuracy is defined relative to the angular resolution step size, targeting heading estimates within $\pm 0.25°$.

6.5.2 Project Timeline

The processing steps—including DEM extraction, database generation, and matching—are computationally efficient and fit comfortably within a standard 60-day schedule. The primary resource allocation is directed toward downloading and pre-processing the Street View validation dataset, fine-tuning the segmentation model, and executing the matching evaluation. To prevent API rate-limiting issues, a representative subset of several hundred high-resolution panoramas will be processed, yielding a validation database of several thousand test images.

6.6 Conclusion

The computational, storage, and data requirements of the proposed system are manageable.

REFERENCES

[1] Z. Pan, J. Tang, T. Tjahjadi, X. Xiao, and Z. Wu, "Camera geolocation using digital elevation models in hilly area," *Applied Sciences*, vol. 10, no. 19, p. 6661, 2020.

[2] C. R. Steger, B. Steger, and C. Schär, "HORAYZON v1.2: an efficient and flexible ray-tracing algorithm to compute horizon and sky view factor," *Geoscientific Model Development*, vol. 15, no. 17, pp. 6817–6840, 2022.

[3] G. Yang, M. Wang, Q. Zhou, and J. Li, "YUNet: Improved YOLOv11 network for skyline detection," *arXiv preprint arXiv:2502.12449*, 2025.

[4] J. Brejcha and M. Čadík, "GeoPose3K: Mountain landscape dataset for camera pose estimation in outdoor environments," *Image and Vision Computing*, vol. 66, pp. 1–14, 2017.

[5] S. Franks and R. Rengarajan, "Evaluation of Copernicus DEM and comparison to the DEM used for landsat collection-2 processing," *Remote Sensing*, vol. 15, no. 10, p. 2509, 2023.

[6] J. Tomesek, M. Cadik, and J. Brejcha, "CrossLocate: Cross-modal large-scale visual geo-localization in natural environments using rendered modalities," https://github.com/JanTomesek/CrossLocate, 2022.

[7] T. Rakthanmanon, B. Campana, A. Mueen *et al.*, "Data mining a trillion time series subsequences under dynamic time warping," in *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 2013, pp. 3047–3053.

[8] A. Mueen, S. Zhong, Y. Zhu, M. Yeh, K. Kamgar, K. Viswanathan, C. Gupta, and E. Keogh, "The fastest similarity search algorithm for time series subsequences under euclidean distance," http://www.cs.unm.edu/~mueen/FastestSimilaritySearch.html, 2022.

[9] Advanced Online Insights, "A daring plan to capture everest | street view stories," https://www.advancedonlineinsights.com/a-daring-plan-to-capture-everest-street-view-stories/, 2015.

[10] S. Trevisani, T. N. Skrypitsyna, and I. V. Florinsky, "Global digital elevation models for terrain morphology analysis in mountain environments: insights on Copernicus GLO-30 and ALOS AW3D30 for a large Alpine area," *Environmental Earth Sciences*, 2023.

[11] L. Guo, J. Li, A. Dehecq, Z. Li, X. Li, and J. Zhu, "A new inventory of High Mountain Asia surging glaciers derived from multiple elevation datasets since the

1970s," *Earth System Science Data*, vol. 15, no. 7, pp. 2841–2861, 2023.

[12] N. S. H. N. Azmin *et al.*, "Evaluation of global digital elevation model (GDEM) over peninsular Malaysia," *International Journal of Geoinformatics*, 2024, tODO: confirm exact volume/issue/pages.

[13] E. Keogh, "The UCR matrix profile page," https://www.cs.ucr.edu/~eamonn/MatrixProfile.html, 2017, university of California, Riverside.

[14] Ready Tensor, "Distance profile for time-step classification in time series analysis," https://app.readytensor.ai/publications/distance-profile-for-timestep-classification-in-time-series-analysis-ljGAbBceZbpv, 2024.

[15] H. Li, J. Liu, Z. Yang, R. W. Liu, K. Wu, and Y. Wan, "Adaptively constrained dynamic time warping for time series classification and clustering," *Information Sciences*, vol. 534, pp. 97–116, 2020.

[16] H. Yu, M. Che, H. Yu, and Y. Ma, "Research on weed identification in soybean fields based on the lightweight segmentation model DCSAnet," *Frontiers in Plant Science*, vol. 14, p. 1268218, 2023.

[17] NovelioPI, "NovelioPI/horizon-detection-for-uav," https://github.com/NovelioPI/horizon-detection-for-uav, 2021.